Nithish Pizzeria

Full-Stack DevOps Project

Complete Build, Deploy & CI/CD Guide

Stack	React · Node.js/Express · PostgreSQL
Infra	AWS EC2 (Amazon Linux 2023) · Docker Compose
CI/CD	Jenkins · GitHub Webhooks
Email	Nodemailer · Mailtrap SMTP Sandbox

Table of Contents

1. Project Overview & Architecture

Nithish Pizzeria is a fully functional food-ordering web application built as a 3-tier architecture and deployed on AWS EC2 using Docker Compose. A Jenkins CI/CD pipeline automates every deployment on a Git push.

Tier	Technology	Responsibility
Tier 1 — Frontend	React (CRA)	UI, cart, checkout, order confirmation
Tier 2 — Backend	Node.js + Express	REST API, business logic, email
Tier 3 — Database	PostgreSQL 16	menu_items, orders, order_items

Request Flow

Browser → React (port 3000) → Express API (port 5000) → PostgreSQL (port 5432). On order placement, the API saves to the database, then sends a confirmation email via Nodemailer to the customer's Mailtrap sandbox inbox.

2. Frontend — React Application

Key Pages & Components

Page / Component	Path	Purpose
Home	src/pages/Home.js	Hero, highlights section
Menu	src/pages/Menu.js	Live menu from API + category filter
Cart	src/pages/Cart.js	Cart management, quantity controls
Checkout	src/pages/Checkout.js	Order form, validation, API POST
OrderConfirmation	src/pages/OrderConfirmation.js	Order slip, email status
Navbar	src/components/Navbar.js	Sticky nav + cart badge
CartContext	src/context/CartContext.js	Global cart state (useReducer)
API client	src/api/client.js	Centralised fetch helpers

Environment Variable

The frontend uses one environment variable, set in docker-compose.yml:

```
REACT_APP_API_URL=http://<YOUR_ELASTIC_IP>:5000
```

■■ *REACT_APP_* variables are baked into the CRA dev server at startup, not at build time in this setup. If you change this value you must recreate (not just restart) the pizzeria-web container for the new URL to take effect.*

3. Backend — Node.js / Express API

API Endpoints

Method	Endpoint	Description
GET	/api/health	Health check — returns status + timestamp
GET	/api/menu	List all available menu items from DB
POST	/api/orders	Create order, save to DB, send email
GET	/api/orders/:id	Fetch a single order by ID

POST /api/orders — Request Body

```
{
  "customerName":    "Jane Doe",
  "customerEmail":   "jane@example.com",
  "customerPhone":   "555-0100",
  "deliveryAddress": "123 Main St",
  "notes":           "Ring the bell twice",
  "items": [
    { "menuItemId": 1, "quantity": 2 },
    { "menuItemId": 3, "quantity": 1 }
  ]
}
```

■■ *Prices are always looked up server-side from the database. The client never controls the order total.*

File Structure

```
server/
  src/
    index.js              # Express app entry point
    routes/
      menuRoutes.js
      orderRoutes.js
    controllers/
      menuController.js
      ordersController.js # DB transaction + email trigger
    db/
      pool.js             # pg connection pool
      schema.sql          # tables + seed data
    utils/
      email.js            # Nodemailer transporter
  package.json
  Dockerfile
  .env.example
```

4. Database — PostgreSQL Schema

Tables

```
menu_items
  id, name, description, price, category,
  image_url, is_available, created_at

orders
  id, customer_name, customer_email, customer_phone,
  delivery_address, status, total_amount, notes, created_at

order_items
  id, order_id (FK), menu_item_id (FK),
  item_name, unit_price, quantity
```

Key design decisions

• order_items snapshots item_name and unit_price at the time of ordering — so if the menu changes later, historical orders remain accurate.

• orders and order_items are created inside a single Postgres transaction — either both succeed or neither does.

• schema.sql is auto-run on first container start via Docker's /docker-entrypoint-initdb.d/ mechanism. It only seeds if the table is empty.

• status column uses a CHECK constraint to enforce valid values: pending, confirmed, preparing, out_for_delivery, delivered, cancelled.

5. Docker & Docker Compose Setup

Services in docker-compose.yml

Service	Image / Build	Port	Role
db	postgres:16-alpine	5432	PostgreSQL database
api	Build: ./server	5000	Express REST API
web	Build: . (root)	3000	React frontend

Important Docker commands

Start full stack (detached):

```
docker compose -p pizzeria up --build -d
```

Stop and remove containers:

```
docker compose -p pizzeria down
```

View running containers:

```
docker ps
```

View API logs:

```
docker logs pizzeria-api --tail 50
```

Check env inside container:

```
docker exec pizzeria-api env | grep SMTP
```

Remove a stuck container:

```
docker rm -f pizzeria-db
```

■■ *Always use -p pizzeria with docker compose commands to ensure a consistent project name across Jenkins and manual runs. Without it, containers from different working directories will conflict.*

6. AWS EC2 Instance Setup

Instance Configuration

Setting	Value
AMI	Amazon Linux 2023
Instance type	t3.medium (2 vCPU, 4 GB RAM)
Storage	15 GiB gp3
Key pair	.pem file — download once, store safely
Elastic IP	Allocated and associated to prevent IP change on reboot

Security Group Inbound Rules

Type	Port	Source	Purpose
SSH	22	My IP	SSH access
Custom TCP	3000	Anywhere	React frontend
Custom TCP	5000	Anywhere	Express API
Custom TCP	8080	My IP	Jenkins dashboard

SSH Connection

```
ssh -i C:\Keys\linux.pem ec2-user@<ELASTIC_IP>
```

■■ *Default user for Amazon Linux is ec2-user, not ubuntu.*

7. Deploying the Application

Install Docker on Amazon Linux 2023

```
sudo dnf update -y
sudo dnf install -y docker
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
# Log out and back in, then install Compose plugin:
sudo mkdir -p /usr/local/lib/docker/cli-plugins
sudo curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86_64 \
     -o /usr/local/lib/docker/cli-plugins/docker-compose
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
docker compose version  # verify
```

Clone and configure

```
git clone https://github.com/<your-username>/<repo-name>.git
cd <repo-name>
cp .env.example .env
nano .env   # fill in DB credentials and SMTP values
```

First-time launch

```
docker compose -p pizzeria up --build -d
```

■■ *Postgres auto-runs schema.sql on first start, creating all tables and seeding the menu. Subsequent starts skip the seed since the table is already populated.*

8. Jenkins CI/CD Pipeline

Install Jenkins on Amazon Linux 2023

```
sudo dnf install -y java-21-amazon-corretto
sudo wget -O /etc/yum.repos.d/jenkins.repo \
     https://pkg.jenkins.io/redhat-stable/jenkins.repo
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
sudo dnf install -y jenkins
sudo systemctl enable --now jenkins
# Give Jenkins permission to run Docker:
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
```

Initial unlock

Open http://<ELASTIC_IP>:8080 in your browser. Get the admin password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Paste it in the browser, click Install suggested plugins, then create an admin user.

Jenkinsfile (in repo root)

```
pipeline {
    agent any
    stages {
        stage("Checkout") {
            steps { checkout scm }
        }
        stage("Build & Deploy") {
            steps {
                withCredentials([file(
                    credentialsId: "pizzeria-env-file",
                    variable: "ENV_FILE"
                )]) {
                    dir("${WORKSPACE}") {
                        sh """
                            rm -f .env
                            cp "$ENV_FILE" .env
                            chmod 600 .env
                            docker compose -p pizzeria down || true
                            docker compose -p pizzeria up --build -d
                        """
                    }
                }
            }
        }
        stage("Health Check") {
            steps {
                sh """
                    for i in {1..12}; do
```

```
                        curl -f http://localhost:5000/api/health && exit 0
                        sleep 5
                    done
                    exit 1
                """
            }
        }
    }
    post {
        success { sh "docker image prune -f"; echo "Deployed!" }
        failure { echo "Deployment failed." }
    }
}
```

Secrets management

The .env file is never committed to git. Instead, it is stored in Jenkins as a Secret File credential with ID pizzeria-env-file.

Manage Jenkins → Credentials → System → Global credentials → Add Credentials → Kind: Secret file → Upload your local .env → ID: pizzeria-env-file

> ■■ *Jenkins automatically masks the file path in all console output so secrets are never exposed in build logs.*

9. GitHub Webhook Integration

A webhook tells GitHub to notify Jenkins automatically every time you push, so the pipeline runs without clicking Build Now.

Step 1 — Enable GitHub hook trigger in Jenkins job

Open your pizzeria-pipeline job → Configure → Build Triggers → check GitHub hook trigger for GITScm polling → Save.

Step 2 — Add webhook in GitHub

• Go to your GitHub repo → Settings → Webhooks → Add webhook.

• Payload URL: http://<ELASTIC_IP>:8080/github-webhook/

• Content type: application/json

• Which events: Just the push event

• Active: checked

• Click Add webhook.

> ■■ *Port 8080 must be open in your EC2 security group to GitHub's IP ranges for webhooks to reach Jenkins. If you restricted 8080 to My IP only, change it to Anywhere (0.0.0.0/0) or add GitHub's published IP ranges.*

10. Email Integration (Nodemailer + Mailtrap)

How it works

When POST /api/orders is called, after the DB transaction commits, ordersController.js calls sendOrderConfirmationEmail(). This function uses Nodemailer to send an HTML-formatted order receipt to the customer's email address. If SMTP is not configured, the email is logged to the console instead of sent — the order still succeeds and the API response includes email.sent: false.

Mailtrap sandbox setup

• Sign up at https://mailtrap.io (free).

• Go to Sandboxes in the left sidebar.

• Click My Sandbox → Integration tab → SMTP credentials.

• Copy Host, Port, Username, Password into your .env file.

• Sent emails appear in the Mailtrap inbox instantly for inspection.

SMTP .env values for Mailtrap

```
SMTP_HOST=sandbox.smtp.mailtrap.io
SMTP_PORT=587
SMTP_USER=<your mailtrap username>
SMTP_PASSWORD=<your mailtrap password>
SMTP_FROM="Nithish Pizzeria <no-reply@nithishpizzeria.com>"
```

11. Common Issues & Fixes

Failed to fetch / ERR_CONNECTION_REFUSED on Menu page

The frontend is trying localhost:5000. REACT_APP_API_URL in docker-compose.yml is set to localhost instead of your Elastic IP. Update it to http://<ELASTIC_IP>:5000, commit, push, and rebuild.

Order placed but no email received

SMTP credentials are blank in the running container. Check with: docker exec pizzeria-api env | grep SMTP. If blank, the container was started before .env was set. Run: docker compose -p pizzeria down && docker compose -p pizzeria up --build -d

Conflict: container name pizzeria-db already in use

A stale container from a previous run is still registered. Run: docker rm -f pizzeria-db pizzeria-api pizzeria-web, then docker compose up again.

cp: cannot create regular file .env: Permission denied (Jenkins)

A stale .env in the Jenkins workspace has wrong ownership. Run: sudo rm -f /var/lib/jenkins/workspace/pizzeria-pipeline/.env && sudo chown -R jenkins:jenkins /var/lib/jenkins/workspace/pizzeria-pipeline

compose build requires buildx 0.17.0 or later

Install Docker Buildx system-wide: sudo mkdir -p /usr/local/lib/docker/cli-plugins && sudo curl -SL [buildx latest URL] -o /usr/local/lib/docker/cli-plugins/docker-buildx && sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-buildx

EC2 public IP changed after restart

This happens if you don't have an Elastic IP. Allocate one in EC2 → Network & Security → Elastic IPs and associate it with your instance. Then update REACT_APP_API_URL and CORS_ORIGIN in docker-compose.yml.

SMTP Host not available (Gmail App Passwords)

Google blocks App Passwords if 2FA is not enabled, or for some account types. Use Mailtrap sandbox instead for testing — it requires no domain verification.

12. Environment Variables Reference

Root .env (read by docker-compose.yml)

Variable	Example Value	Description
`DB_USER`	pizzeria_user	Postgres username
`DB_PASSWORD`	somepassword123	Postgres password
`DB_NAME`	pizzeria_db	Postgres database name
`SMTP_HOST`	sandbox.smtp.mailtrap.io	SMTP server hostname
`SMTP_PORT`	587	SMTP port (587 for STARTTLS)
`SMTP_USER`	1fec3c7791227c	SMTP username
`SMTP_PASSWORD`	your_password	SMTP password / app password
`SMTP_FROM`	"Nithish Pizzeria..."	From address in emails

Frontend variable (docker-compose.yml web service)

Variable	Example	Description
`REACT_APP_API_URL`	http://3.24.192.178:5000	Backend API base URL (must use public IP, not localhost)

■■ *Never commit .env to git. It contains real credentials. The .gitignore in this project already excludes it — verify with: git status --short (you should NOT see .env listed).*